A Fixed Iterated Greedy Sequence Cuts Weighted Tardiness on an Eight-Stage Flow Line by More than 2,500 Weighted Minutes per Period at Every Level of Processing-Time Variation Tested

OM 522 Team Project, Option 6

Muting (Don) Ma

University of Alabama, Fall 2026

November 2, 2026

Abstract. On a flow line where jobs cannot pass one another, a production planner must commit to one job order for every stage, and the earliest due date rule ignores job weights, release times, and the processing times at later stages. We applied the iterated greedy algorithm that Ruiz and Stützle proposed for flow shops with sequence-dependent setup times to an eight-stage line without setups but with release times and weighted due dates, and compared its fixed sequence with the earliest due date sequence on exact data and on 180 paired scenarios of random processing times. On development instances with 15, 45, and 120 jobs, iterated greedy lowered total weighted tardiness by 98.5, 97.6, and 81.0 percent, and a mixed-integer program solved with HiGHS proved its values optimal on the 6-job and 15-job instances. When each processing time of the 45-job instance varied with a coefficient of variation of 0.2, the iterated greedy plan had the lower weighted tardiness in all 30 scenarios, with a mean of 420.6 weighted minutes against 3,541.1, an 88.1 percent reduction. As the coefficient of variation rose from 0.1 to 0.8, the relative reduction fell from 94.6 to 27.2 percent while the mean absolute reduction stayed between 2,572.0 and 3,683.7 weighted minutes per period, and at 0.8 the 10th percentile of the per-period reduction fell to 851.5 and the earliest due date plan won 1 of 30 scenarios. A planner can therefore adopt the searched sequence for its absolute gain and, on the evidence of the one factor we varied, should expect that gain to arrive less dependably in a single period as processing times grow less predictable.

Contents

1 Background **2**

2 Data **3**

3 Objective **4**

4 Model formulation **5**

4.1 One sequence fixes every start time 5

4.2 A mixed-integer program defines the optimum 5

4.3 The rule sorts once, and iterated greedy searches by insertion 6

5 Experiment **8**

5.1 Both methods ran on the same instances with fixed settings 8

5.2 HiGHS certified optimal values on the two smallest instances 8

5.3 Thirty saved scenarios tested the fixed plans 8

References **9**

1. Background

On a flow line such as printed circuit board assembly or furniture finishing, every job visits the same stages in the same order, and when jobs cannot pass one another, the order chosen at the first stage holds at every stage [1]. On the line studied here operations take 2 to 20 minutes, so a job that needs 2 minutes at the first stage and 20 at a later one would delay every job behind it at that later stage, and the waiting would build up downstream. The planner chooses one sequence for a period’s jobs, which are released at different times and carry due dates of different importance, expressed as weights. Vallada, Ruiz, and Minella report that dispatching rules are used very often in practice and that earliest due date is among the most common rules for total tardiness [2], and the course brief for this line makes it the rule that a researched method must beat [1]. Earliest due date orders the jobs by due date alone, so it cannot see a job’s weight, its release time, or its processing times at the stages after the first.

Minimizing total tardiness on a permutation flow shop is NP-hard in the ordinary sense with one machine and in the strong sense with two or more [2], and on this line an exact solver needed 85.8 seconds to certify the optimum for only 15 jobs, so a planner sequencing 45 to 120 jobs per period needs a heuristic. Ruiz and Stützle’s iterated greedy algorithm repeatedly removes a few jobs from a sequence and reinserts each at its best position [3], and on flow shops with sequence-dependent setup times and the weighted tardiness objective it had the lowest average deviation from the best known solutions among the methods they tested [4]. In a review of 40 methods for total tardiness, the heuristics built on inserting and interchanging jobs performed best among the heuristics, ahead of every dispatching rule [2]. These evaluations used instances with fixed processing times and no release times, and none of them executed a searched sequence on processing times that differ from the plan. A sequence tuned to the exact times in a data file could lose its advantage once the 2- to 20-minute operations run long or short, and a planner who must fix the order in advance cannot tell from these studies whether it does.

We implemented the earliest due date rule exactly as the course brief states it [1] and applied iterated greedy with local search [4] to an eight-stage line with release times and no setup times. A mixed-integer program solved with HiGHS [5] certified the optimal values of the two smallest instances, and a checker that shares no code with either method scored every schedule. We then held each method’s sequence fixed and executed it on 30 scenarios in which every processing time was multiplied by an independent lognormal factor with mean 1 and a coefficient of variation of 0.2, and on 30 further scenarios at each of five levels of variation from 0.1 to 0.8.

On exact data, iterated greedy left 1.5, 2.4, and 19.0 percent of the rule’s weighted tardiness on the instances with 15, 45, and 120 jobs, and it matched the proven optimum on the 6-job and 15-job instances. Held fixed on the 45-job instance at a coefficient of variation of 0.2, its plan left 11.5 jobs late per period on average, against 36.1 for the rule’s plan, and across all six sets of scenarios it had the lower weighted tardiness in 179 of 180. The mean reduction stayed between 2,572.0 and 3,683.7 weighted minutes at every level of variation, while the 10th percentile of the reduction in a single period fell from at least 1,781.1 weighted minutes at coefficients up to 0.5 to 851.5 at 0.8.

A planner can adopt the searched sequence for an absolute gain that held at every level of variation we tested, at the cost of a sequence that cannot be traced by hand and seconds of computation. Processing-time variability is the one condition we varied, and because it eroded the gain in the worst periods first, it is the first quantity to measure on a line before relying on the plan.

2. Data

The data are synthetic instances that the course instructor generated for option 6 of the OM 522 team project, with all times in minutes [1]. Each instance consists of a file of jobs, giving each job's release time, due date, and weight, a file of stages, giving their order on the line, and a file with one positive processing time for every job and stage. Table 1 summarizes the five instances: a 6-job case on 4 stages meant for tracing by hand, development instances with 15, 45, and 120 jobs on 8 stages, and a 10-job edge case on 4 stages.

Table 1: The five supplied instances. Processing times are per job and stage. The due-date window is the time from a job's release to its due date, and its median is taken over the instance's jobs. All times are in minutes.

Instance	Jobs	Stages	Processing times	Releases	Median due-date window	Weights
hand_checkable	6	4	3 to 19	8 to 78	30	1, 2, 4
development_small	15	8	2 to 20	24 to 210	158	1, 2, 4, 5
development_medium	45	8	2 to 20	10 to 607	154	1 to 5
development_large	120	8	2 to 20	3 to 1,690	154	1 to 5
edge_case	10	4	0 to 20	5 to 126	87.5	1, 2, 3, 5

The three development instances share one generator, so their processing times range from 2 to 20 minutes and average between 85.1 and 88.8 minutes of total processing per job over the 8 stages. Their releases spread over 186, 597, and 1,687 minutes, which gives arrival rates between 0.071 and 0.081 jobs per minute, so the larger instances cover longer planning periods at a similar density. The median time from release to due date is 154 to 158 minutes, and subtracting each job's own total processing time leaves a median of 64.5 to 74 minutes that a job can spend waiting before it is late. Weights are integer priority scores from 1 to 5, and a weighted minute is one minute of lateness multiplied by the job's priority [1].

Before solving, our tool applies the four checks the project guide requires: every required file and column is present, every value has its declared type, every job and stage referenced by a processing time exists, and every processing time is positive [6]. It reports each failure with its file, column, and line, and it never replaces a missing or invalid value. The edge case contains a processing time of zero, so the tool rejects it without writing a decision, as the guide specifies for this option [6].

3. Objective

A job's tardiness is the time by which its completion at the last stage exceeds its due date, and it is zero when the job finishes on time. The primary measure is total weighted tardiness, the sum over jobs of weight times tardiness, in weighted minutes [1]. Due dates are soft targets, so a late job makes a schedule worse without making it infeasible [1].

Total waiting sums, over jobs and over every stage after the first, the time from a job's completion at one stage to its start at the next [1]. The brief allows unlimited waiting space between stages, so waiting measures work in process piling up on the line and carries no feasibility consequence. Every comparison also reports the computation time of each method.

Iterated greedy compares sequences by total weighted tardiness and breaks exact ties by total waiting, because many sequences can share a tardiness value when most jobs finish on time. The earliest due date rule optimizes nothing; it sorts the jobs once. The mixed-integer program minimizes total weighted tardiness alone. Every reported value of either measure is recomputed after the fact from the exported schedule, so no method reports its own score.

4. Model formulation

4.1 One sequence fixes every start time

The line processes n jobs, indexed by j, on m stages, indexed by k in line order. Job j is released at time r_j, is due at d_j, carries weight w_j, and needs processing time $p_{jk} > 0$ at stage k. A sequence π lists the jobs in the order they enter the line, with $\pi(i)$ the job in position i, and the same order holds at every stage. Given π, each operation starts at the later of the job's completion at the previous stage and the stage's completion of the previous job:

$$S_{\pi(i),k} = \max\bigl\{C_{\pi(i),k-1},\ C_{\pi(i-1),k}\bigr\}, \qquad C_{\pi(i),k} = S_{\pi(i),k} + p_{\pi(i),k}, \tag{4.1}$$

with the conventions $C_{\pi(i),0} = r_{\pi(i)}$, so the first stage waits for the release, and $C_{\pi(0),k} = -\infty$, so the first job waits for no other.

The tardiness of job j is $T_j = \max\{0,\ C_{jm} - d_j\}$, and the two measures of section 3 are

$$Z(\pi) = \sum_{j=1}^{n} w_j T_j, \qquad W(\pi) = \sum_{j=1}^{n} \sum_{k=2}^{m} \bigl(S_{jk} - C_{j,k-1}\bigr). \tag{4.2}$$

By induction over positions and stages, any schedule that keeps sequence π and satisfies the brief's constraints starts each operation no earlier than (4.1), so the recursion gives every job its earliest completion at the last stage and π its smallest value of Z. The planning problem is therefore to choose the sequence π that minimizes $Z(\pi)$, and (4.1) is also the execution rule that the brief prescribes for the stress test [1].

4.2 A mixed-integer program defines the optimum

To certify optimal values, we state the problem as a mixed-integer program over sequence positions $q = 1, \dots, n$. The binary variable x_{jq} equals 1 when job j takes position q, B_{qk} and E_{qk} are the start and end of the job in position q at stage k, and $T_j \ge 0$ is the tardiness of job j:

$$\min \quad \sum_{j=1}^{n} w_j T_j \tag{4.3}$$

$$\text{s.t.} \quad \sum_{q=1}^{n} x_{jq} = 1 \quad \forall j, \qquad \sum_{j=1}^{n} x_{jq} = 1 \quad \forall q, \tag{4.4}$$

$$E_{qk} = B_{qk} + \sum_{j=1}^{n} p_{jk}\, x_{jq} \quad \forall q, k, \tag{4.5}$$

$$B_{q1} \ge \sum_{j=1}^{n} r_j\, x_{jq} \quad \forall q, \tag{4.6}$$

$$B_{qk} \ge E_{q,k-1} \quad \forall q,\ k \ge 2, \tag{4.7}$$

$$B_{qk} \ge E_{q-1,k} \quad \forall q \ge 2,\ k, \tag{4.8}$$

$$T_j \ge E_{qm} - d_j - U_j\,(1 - x_{jq}) \quad \forall j, q, \tag{4.9}$$

$$x_{jq} \in \{0, 1\}, \quad T_j \ge 0.$$

Constraints (4.4) place every job in exactly one position, (4.5) runs each operation without interruption, (4.6) and (4.7) enforce release and stage order, and (4.8) lets a stage start a job only after finishing the previous position, which also keeps one order at every stage. Constraint (4.9) charges the tardiness of the job in position q to that job, and it is inactive for every other job when $U_j = \max\{0,\ \max_h r_h +$

$\sum_{h,k} p_{hk} - d_j\}$, with h running over all jobs.

Under recursion (4.1), every job of every sequence finishes by $\max_h r_h + \sum_{h,k} p_{hk}$, because a completion time is a release plus processing times along one path through the schedule. The bound U_j therefore never forces a positive tardiness on the earliest-start schedule of any sequence, which by section 4.1 is the best schedule for that sequence. The program's optimal value equals $\min_\pi Z(\pi)$, and we use it only as a benchmark, never as one of the two compared methods.

4.3 The rule sorts once, and iterated greedy searches by insertion

The earliest due date rule sorts the jobs by d_j, breaks ties by job identifier, and times the resulting sequence by (4.1), exactly as the course brief defines it [1].

Our researched method is IG_RS_LS, the iterated greedy algorithm with local search that Ruiz and Stützle proposed for the flow shop with sequence-dependent setup times and the makespan or weighted tardiness objective [4]; our line has no setup times, so we apply its weighted tardiness version with every setup time equal to zero, as shown in Algorithm 1. It compares sequences by the pair $F(\pi) = (Z(\pi), W(\pi))$ in lexicographic order, and the best position for a job is the position in a partial sequence that gives the smallest F, the earliest one on an exact tie. The start, NEH_EWDD, orders the jobs by ascending d_j/w_j and inserts them one at a time at their best positions [4], following the insertion steps of NEH as stated by Ruiz and Stützle [3]. Each iteration removes $\delta = 4$ jobs at random, reinserts them in the order removed, and applies a local search that moves single jobs to their best positions until no move improves F. A worse sequence replaces the current one with probability $\exp(-\Delta/\Theta)$, where Δ is its increase in Z and $\Theta = \lambda \sum_{j,k} p_{jk}/(10\, n\, m)$ with $\lambda = 0.5$, the constant-temperature rule and settings of the source [4].

We break ties in Z by waiting, because the brief names waiting as the secondary measure and the source compares on one objective. We start from the earliest due date sequence whenever it beats NEH_EWDD, so the method can never return a sequence worse than the rule; on all four instances NEH_EWDD was the better start. We stop after $L = 5000$ iterations with a fixed seed of 522, where the source stops at a CPU-time limit, because the course requires a rerun on another computer to reproduce the submitted result [6]. The best-position search times the jobs ahead of each position once and abandons a position as soon as its running pair (Z, W) can no longer beat the best position found, which is safe because Z and W only grow as jobs are appended.

Algorithm 1 Iterated greedy for weighted tardiness, after Ruiz and Stützle [4]. $F = (Z, W)$ is compared lexicographically, and every insertion goes to the best position.

Require: instance; destruction size $\delta = 4$; temperature factor $\lambda = 0.5$; iteration limit $L = 5000$; seed 522

$\pi \leftarrow$ NEH_EWDD: jobs by ascending d_j / w_j, each inserted at its best position
if $F(\pi^{\mathrm{EDD}}) < F(\pi)$ **then** $\pi \leftarrow \pi^{\mathrm{EDD}}$ ▷ never start worse than the rule
end if
$\pi \leftarrow \textsc{LocalSearch}(\pi)$; $\pi^{*} \leftarrow \pi$; $\Theta \leftarrow \lambda \sum_{j,k} p_{jk} / (10\, n\, m)$
for $\ell = 1, \ldots, L$ **do**
    remove δ jobs chosen at random from π, keeping the removed jobs in order of removal
    insert each removed job, in that order, at its best position
    $\pi' \leftarrow$ LocalSearch of the rebuilt sequence
    if $F(\pi') < F(\pi)$ **then** $\pi \leftarrow \pi'$; **if** $F(\pi) < F(\pi^{*})$ **then** $\pi^{*} \leftarrow \pi$
    else if $u \le \exp\!\big(-(Z(\pi') - Z(\pi))/\Theta\big)$ for u uniform on $[0, 1]$ **then** $\pi \leftarrow \pi'$
    end if
end for
return π^{*}

function $\textsc{LocalSearch}(\pi)$
    repeat
        for each job j, in random order **do**
            remove j and insert it at its best position, giving π''
            if $F(\pi'') < F(\pi)$ **then** $\pi \leftarrow \pi''$
            end if
        end for
    until a full pass changes nothing
    return π
end function

5. Experiment

5.1 Both methods ran on the same instances with fixed settings

Both methods ran on the hand-checkable instance and the three development instances, with the same input files and the measure definitions of section 3. Iterated greedy used its default settings of $L = 5000$ iterations and seed 522, with a safety cap of 300 seconds that no run reached, and we repeated it with seeds 1 to 5 to see how much the result depends on the seed. NEH_EWDD alone, the first step of Algorithm 1, appears in the comparison as a construction-only reference, so the share of the improvement due to the search can be read off. All runs used one laptop with an Intel Core Ultra 7 155U processor under Linux, Python 3.12, NumPy 2.5.4, and Numba 0.68.0, and the computation time covers the method alone, after one warm-up run that compiles the timing code. We expected iterated greedy to have the lower total weighted tardiness on every instance, because it evaluates the actual tardiness of each candidate sequence while the rule never does.

Each method's sequence was timed by (4.1) and exported as a list of start and completion times for every job and stage. A separate checker, written without any code from the methods, read only the input files and the exported schedule, confirmed each constraint of the brief within a tolerance of 10^{-8} minutes, and recomputed Z, W, and the share of jobs finishing on time. Every number we report for a schedule comes from this checker.

5.2 HiGHS certified optimal values on the two smallest instances

We solved program (4.3) to (4.9) with HiGHS 1.15.1 [5] on the 6-job and 15-job instances, using one thread, a relative gap tolerance of zero, and a limit of 600 seconds. The job order of each solution was re-timed by (4.1) and confirmed by the checker, which supplied its Z and W. We did not run the program on the larger instances, because the 15-job instance already needed 85.8 seconds and more than 15,000 branch-and-bound nodes.

5.3 Thirty saved scenarios tested the fixed plans

The uncertain input is processing time, as the brief specifies [1]. In scenario ω, each processing time becomes $p_{jk}\,\varepsilon_{jk\omega}$, where the factors $\varepsilon_{jk\omega}$ are independent lognormal draws with mean 1 and coefficient of variation ν, obtained with $\sigma^2 = \ln(1+\nu^2)$ and log-mean $-\sigma^2/2$. For the main test we drew 30 scenarios at $\nu = 0.2$ for the 45-job instance from seed 20261103 and saved them to files before either method ran, so both plans faced identical realizations. Each method's plan is its sequence for the exact data under default settings. Each plan kept its sequence in every scenario and was timed by (4.1) with the realized times, the brief's execution rule, and the checker confirmed each realized schedule against that scenario's times.

For each plan we report the mean and the 90th percentile, by linear interpolation, of Z and W over the 30 scenarios, and the number of scenarios in which the plan breaks a hard constraint. To find where the comparison changes, we repeated the execution with 30 new scenarios at each of $\nu = 0.1$, 0.2, 0.3, 0.5, and 0.8, drawn from seeds 6522 to 6526. We expected both plans to grow later as ν grows and the iterated greedy plan to lose part of its advantage, because a plan tuned to exact times has more on-time jobs that a delay can make late.

References

[1] OM 522 instructor. OM 522 team project, option 6: Sequencing jobs through a common production line. Option brief and data package (schema version 4), University of Alabama, 2026. Course document, Fall 2026.

[2] Eva Vallada, Rubén Ruiz, and Gerardo Minella. Minimising total tardiness in the m-machine flowshop problem: A review and evaluation of heuristics and metaheuristics. *Computers & Operations Research*, 35(4):1350-1373, 2008. doi:10.1016/j.cor.2006.08.016.

[3] Rubén Ruiz and Thomas Stützle. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research*, 177(3):2033-2049, 2007. doi:10.1016/j.ejor.2005.12.009.

[4] Rubén Ruiz and Thomas Stützle. An Iterated Greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. *European Journal of Operational Research*, 187(3):1143-1159, 2008. doi:10.1016/j.ejor.2006.07.029.

[5] HiGHS development team. HiGHS: High performance software for linear optimization, version 1.15.1, 2026. URL: https://highs.dev.

[6] OM 522 instructor. OM 522 team project guide. Revised October 14, 2026, University of Alabama, 2026. Course document, Fall 2026.